A Minimal Reasoning-Acting Loop for

Abstract. This paper describes a minimal reasoning-acting loop that alternates between deliberation and tool invocation without relying on external orchestration frameworks.

1 Introduction. Most agent frameworks bolt a planner on top of a chat model. We argue that a single loop with strict resource guards is easier to reason about.

2 States. The loop exposes five observable states: think, act, observe, finish and abort, and every transition is emitted as an event so that the caller can render progress.

3 Guards. Four guards keep the loop bounded: a maximum iteration count, a maximum number of consecutive tool failures, a step budget without progress, and a per-turn monetary cost ceiling.

4 Tool timeout. A tool that hangs would otherwise stall the whole turn, so each invocation is wrapped in a timeout; the timeout is deliberately loose because legitimate tools may take tens of seconds.

5 Cancellation. User cancellation must not be recorded as a tool failure, so the loop only catches timeout errors and lets cancellation propagate untouched.

6 Finalization. When the budget is exhausted the loop still produces an answer: it calls the model with no tools at all, which structurally guarantees convergence.

Table 1 Default configuration. iterations 12; tool failures 3; token budget 8000; timeout 120 s.

7 Self reflection. On an aborted turn only, the loop may ask the model to critique its own draft and regenerate once with the critique attached.

8 Conclusion. Bounded loops with explicit failure semantics are preferable to unbounded autonomy, especially for single-user local deployments.

9 Observability. Every iteration emits a structured event with the current step index, t

he tool that is about to run and the reason the loop selected it. Without these events a stalled turn is indistinguishable from a slow one.

10 Context budget. The prompt sent to the model grows with every tool result, so the loop projects the next request size before issuing it and drops the oldest tool outputs when the projection crosses a trigger line.

11 Token accounting. Character counts are a poor proxy for tokens because the ratio differs by content type; the loop therefore uses the provider reported usage whenever it is available and only falls back to estimation for the first request of a turn.

12 Tool result shaping. A tool may return far more text than the model needs. Truncation is applied to the payload rather than by removing messages, because removing a message would break the pairing between a call and its result.

13 Idempotence. Tools that mutate state must be safe to retry, since a timeout gives the loop no way to know whether the effect already happened. Read-only tools should be preferred whenever the task allows it.

14 Failure taxonomy. The loop distinguishes a tool that failed, a tool that timed out and a tool that returned nothing useful. Each of these leads to a different next step, and conflating them produces loops that retry hopelessly.

15 Evaluation. Loop behaviour is exercised with scripted tool responses so that the control flow is tested deterministically, while the token accounting is measured against a real provider to keep the two layers honest.

16 Limitations. The loop assumes a single writer; concurrent turns on the same session would make the step budget and the context projection ambiguous.

17 Prompt stability. The loop keeps the system prompt fixed across iterations and only appends observations, so that any change in behaviour can be attributed to the new evidence rather than to a rewritten instruction.

18 Cost ceiling. A per-turn monetary ceiling is enforced before each request; when it is reached the loop finalises instead of continuing, which makes the worst case predictable for the caller.

19 Progress detection. Counting iterations is not the same as counting progress. The loop tracks steps that produced no new information and stops when they accumulate, because a loop that keeps trying without learning is indistinguishable from a hang.

20 Tool selection. Tools are described to the model with the same names they are invoked by; a mismatch between the two is a silent failure that only shows up as the model repeatedly asking for a tool that cannot be resolved.

21 Streaming. Intermediate tokens are streamed to the caller while tools run, so that a long turn does not look like a stalled one, but the stream must never leak the raw tool output.